

03 Setting Up Data Sources

GCP Data Engineering - Healthcare Revenue Cycle Management (RCM) - Setting Up Data Sources

1. Introduction to Data Sources

To begin our RCM data lake project on GCP, the foundational step is to establish our data sources. While in a corporate environment these data sources are typically pre-existing and readily available, for the purpose of replicating a real-world scenario, we will create them from scratch.

2. Required Data Sources:

Our project will utilize the following data sources:

- **Hospital A Database:** An Electronic Medical Record (EMR) database for Hospital A.
- **Hospital B Database:** An Electronic Medical Record (EMR) database for Hospital B.
- **Claims CSV Files:** Monthly insurance claims data provided as CSV files.
- **CPT Codes CSV File:** A file containing Current Procedural Terminology (CPT) codes for standardization.

3. Setting Up Cloud SQL Instances (Hospital A & B Databases)

We will use Cloud SQL, specifically MySQL instances, to host our hospital databases.

Steps to Create Cloud SQL Instances:

1. Navigate to **Cloud SQL** in the GCP Console.
2. Click **"Create Instance"**.
3. Choose **MySQL**.
4. Select the **Enterprise** edition and **Development** for the edition type.
5. Select database version **8.0** (or your preferred version).
6. **Instance ID:**
 - For Hospital A: [Provide Instance ID for Hospital A, e.g., hospital-a-db]
 - For Hospital B: [Provide Instance ID for Hospital B, e.g., hospital-b-db]
7. **Password:** Set a strong password (e.g., 12345 for demonstration).
8. **Region:** Choose a single region (e.g., us-central1).
9. **Customize Instance (Show Configuration):**
 - **Machine Configuration:** Select 2 vCPU, 8 GB memory.
 - **Storage:** Set to 20 GB.
 - **Connections:** Allow **Public IP** (for development purposes; in production, use Private IP or authorized networks).
 - **Data Protection:** Enable automatic backups and point-in-time recovery. Consider disabling "Prevent instance deletion" for development to easily tear down resources.
10. Click **"Create Instance"**.
11. Repeat these steps for both Hospital A and Hospital B instances, ensuring unique instance IDs.

Configuring Cloud SQL Instances (Post-Creation):

Once an instance is created and running:

1. **Allow Network Access:**
 - Go to the instance **Overview** page.
 - Navigate to **Connections > Networking**.
 - Click **"Add network"** and enter 0.0.0.0/0 (this allows access from any IP, suitable for local development; in production, specify your company's network IPs).
 - Click **"Done"** and then **"Save"**.
2. **Create Database:**

- Go to the instance **Overview** page.
- Navigate to **Databases**.
- Click **"Create database"**.
- **Database Name:**
 - For Hospital A: `hospital_a_db`
 - For Hospital B: `hospital_b_db`
- Click **"Create"**.

3. Create User Account:

- Go to the instance **Overview** page.
- Navigate to **Users**.
- Click **"Add user account"**.
- **Username:** `myuser`
- **Password:** `mypassword` (or your chosen password)
- Click **"Add"**.

4. Setting Up Cloud SQL Studio and Creating Tables

To create tables and load data into our Cloud SQL databases:

1. Connect via Cloud SQL Studio:

- Go to the instance **Overview** page.
- Click **"Connect using Cloud SQL Studio"**.
- Provide the **Database name** (`hospital_a_db` or `hospital_b_db`), **Username** (`myuser`), and **Password** (`mypassword`).
- Click **"Authenticate"**.

2. Create Tables:

- Within Cloud SQL Studio, paste the DDL (Data Definition Language) scripts for creating the five EMR tables: `departments`, `encounters`, `patients`, `providers`, and `transactions`. (These scripts will be provided separately).
- Execute the scripts. Verify that all five tables are created successfully.

5. Setting Up Cloud Storage (GCS) for Claims & CPT Codes

GCS will serve as our landing layer for flat files and as the storage for our sample data.

Steps to Create a GCS Bucket:

1. Navigate to **Cloud Storage** in the GCP Console.
2. Click **"Create Bucket"**.
3. **Name your bucket:** Choose a unique name (e.g., `healthcare-bucket-YYYYMMDD`).
4. Select a **region** (e.g., `us-central1`).
5. Choose a **storage class** (e.g., `Standard`).
6. Click **"Create"**.

Creating Folders within the GCS Bucket:

Within your newly created bucket, create the following folders:

- **data** : To store sample EMR CSV files (e.g., `hospital_a` and `hospital_b` subfolders).
- **configs** : To store metadata configuration files.
- **landing** : To serve as the initial landing zone for extracted data from databases and new claims files.
- **temp** : For temporary processing files.
- **claims** : Specifically for monthly claims CSV files (e.g., `hospital_1_claims.csv`, `hospital_2_claims.csv`).
- **cpt_codes** : For the constant CPT codes CSV file.

6. Loading Data into Cloud SQL Tables (from GCS)

To populate the Cloud SQL tables with sample EMR data:

1. Upload Sample EMR Data to GCS:

- Within your GCS bucket, navigate to `data/hospital_a/` and upload the five EMR CSV files for Hospital A (e.g., `departments.csv`, `encounters.csv`, `patients.csv`, `providers.csv`, `transactions.csv`).
- Repeat for `data/hospital_b/` with Hospital B's EMR CSV files.

2. Import Data into Cloud SQL Tables:

- Go to the **Overview** page of your Cloud SQL instance (e.g., Hospital A).
- Click **"Import"**.
- **Source:** Select your GCS bucket and navigate to the respective CSV file (e.g., `gs://your-bucket-name/data/hospital_a/departments.csv`).
- **Format:** Select **CSV**.
- **Destination:**
 - **Database:** Select your database (e.g., `hospital_a_db`).
 - **Table:** Enter the table name (e.g., `departments`).
- Click **"Import"**.
- Repeat this process for all five tables in both Hospital A and Hospital B databases.

7. Understanding Claims and CPT Codes Data

- **Claims Data:** These are typically received monthly from clients (e.g., via email or a drive link) and should be uploaded to the `claims` folder in your GCS bucket. Each file will contain details like `claim_ID`, `transaction_ID`, `patient_ID`, `payer_ID`, `claim_amount`, `paid_amount`, and more.
- **CPT Codes Data:** This is a static reference file. It contains constant codes for medical procedures and diagnoses and generally does not change. Upload this file once to the `cpt_codes` folder in your GCS bucket.

8. Implementing Metadata-Driven Approach (Config File)

A crucial aspect of this project is a metadata-driven approach, which involves using a configuration file to drive pipeline execution.

Config File Structure (Example): The `config.csv` file (to be uploaded to the `configs` folder in GCS) defines how to process each table:

database	datasource	tablename	loadtype	watermark	is_active	targetpath
hospital-a-mysql-db	hospital_a_db	encounters	Incremental	ModifiedDate	1	hospital-a
hospital-a-mysql-db	hospital_a_db	patients	Incremental	ModifiedDate	1	hospital-a
hospital-a-mysql-db	hospital_a_db	transactions	Incremental	ModifiedDate	1	hospital-a
hospital-a-mysql-db	hospital_a_db	providers	Full		1	hospital-a
hospital-a-mysql-db	hospital_a_db	departments	Full		1	hospital-a
hospital-b-mysql-db	hospital_b_db	encounters	Incremental	ModifiedDate	1	hospital-b
hospital-b-mysql-db	hospital_b_db	patients	Incremental	ModifiedDate	1	hospital-b
hospital-b-mysql-db	hospital_b_db	transactions	Incremental	ModifiedDate	1	hospital-b
hospital-b-mysql-db	hospital_b_db	providers	Full		1	hospital-b
hospital-b-mysql-db	hospital_b_db	departments	Full		1	hospital-b

Key Configuration Fields:

- **database** : Specifies the source database identifier (e.g., `hospital-a-mysql-db`, `hospital-b-mysql-db`)
- **datasource** : Identifies the logical data source name (e.g., `hospital_a_db`, `hospital_b_db`)
- **tablename** : The specific table to be processed (e.g., `encounters`, `patients`, `transactions`, `providers`, `departments`)
- **loadtype** : Indicates whether to perform a `Full` load (for reference tables like `departments`, `providers` which change rarely) or `Incremental` load (for transactional tables like `patients`, `encounters`, `transactions` which change frequently)
- **watermark** : For incremental loads, specifies the column to track changes. In this configuration, all incremental tables use `ModifiedDate` as the watermark column

- **is_active** : A flag (1 for active, 0 for inactive) to control whether a table should be processed by the pipeline. This allows for dynamic inclusion/exclusion of tables without code changes
- **targetpath** : Specifies the target path or folder structure for storing the processed data (e.g., `hospital-a` , `hospital-b`)

This metadata will guide our data pipelines, ensuring efficient and controlled data ingestion and processing based on predefined rules. The standardized use of `ModifiedDate` as the watermark column across all incremental loads simplifies pipeline logic and ensures consistent change tracking across different tables and hospitals.